Chaklam Silpasuwanchai, Todsavad Tangtortan Updated 9 March 2026
AT82.03: Machine Learning

A6: Naive RAG vs Contextual Retrieval

In this assignment, you will apply RAG (Retrieval-Augmented Generation) techniques. Instead of a personal chatbot, you will build a domain-specific QA system based on a specific textbook chapter determined by your student ID. You will also implement and compare two retrieval strategies: Naive RAG and Contextual Retrieval.

Note: You are ENCOURAGED to work with your friends, but DISCOURAGED to blindly copy other's work. Both parties will be given 0.

Note: Comments should be provided sufficiently so we know you understand. Failure to do so can raise suspicion of possible copying/plagiarism.

Note: You will be graded upon (1) documentation, (2) experiment, (3) implementation.

Note: This is a one-week assignment, but start early.

Deliverables: The GitHub link containing the jupyter notebook, a README.md of the GitHub, and the folder of your web application called 'app'.

Task 1. Source Discovery & Data Preparation

1) **Select Your Chapter:** Identify the last digit of your student ID (e.g., if your ID is `st124859`, the last digit is **9**). You must extract the content of the corresponding chapter from the course textbook[1] or a relevant machine learning book.
   - If the last digit is 1, use Chapter 11.
   - If the last digit is 0, use Chapter 10.
   - Example: ID ending in 2 → Chapter 2; ID ending in 9 → Chapter 9.
2) **Document Processing:** Load and process the text from your assigned chapter. Clean the data as necessary for RAG implementation. (1 point)
3) **QA Pair Generation:** Create a dataset of at least **20 Question-Answer pairs** based strictly on the content of your assigned chapter.
   - *Collaboration Note:* You may collaborate with other students who have the same assigned chapter to generate these questions, but the implementation must be your own.

   (1 point)

Task 2. Technique Comparison: Naive RAG vs. Contextual Retrieval

1) **Implement Naive RAG:** Build a standard RAG pipeline using a basic chunking strategy and a standard vector retriever.
2) **Implement Contextual Retrieval:** Implement the Contextual Retrieval technique[2] where context is prepended to chunks to improve retrieval quality.

   Hint Code for Contextual Enrichment:

```python
async def enrich_chunk(chunk: str, document: str, title: str) -> str:
    """Add contextual prefix using LLM"""
    prompt = f"""
Title: {title}
{document[:4000]}
{chunk}

Provide brief context (1-2 sentences) explaining what this chunk discusses
in relation to the full document. Format: "This chunk from [title] discusses
    [explanation]." """
    response = await client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
```

[1] https://web.stanford.edu/~jurafsky/slp3/

[2] https://www.anthropic.com/engineering/contextual-retrieval

```
        temperature=0,
        max_tokens=150
    )

    context = response.choices[0].message.content.strip()

    # Embed the contextualized version
    return f"{context}\n\n{chunk}"
```

Example - Before and After:

- **BEFORE:** "Revenue grew 40% to $314M with improved margins."
- **AFTER:** "This chunk from ACME Corp's Q2 2024 SEC filing discusses quarterly financial performance compared to Q1 2024.
  Revenue grew 40% to $314M with improved margins."

3) **Evaluation:** Run your 20 QA pairs through both pipelines.
4) **Analysis:** Calculate the ROUGE scores (ROUGE-1, ROUGE-2, ROUGE-L) for the generated answers against the ground truth. Present your findings in the table format below. Discuss the results. (2 points)

Evaluation Table:

Method	ROUGE-1	ROUGE-2	ROUGE-L
Naive RAG	0.XX	0.XX	0.XX
Contextual Retrieval	0.XX	0.XX	0.XX

Note: RAG utilizes two models: a **retriever** model and a **generator** model. Ensure you clearly state which models you are using for each component.

Task 3. Chatbot Development - Web Application

1) Develop a simple web application (e.g., Streamlit or Chainlit) featuring a chat interface.
2) The chatbot should allow the user to ask questions about the assigned chapter.
3) The backend should utilize the **Contextual Retrieval** method you implemented in Task 2 to generate responses.
4) **Deliverable:** The app must display the generated answer and cite the source chunk used. (0.5 point)

Submission Instructions: Create a folder named `answer` in your repository. Submit your JSON evaluation file inside this folder with the naming convention `response-st-xxxxxx-chapter-x.json` (e.g., `response-st124859-chapter-9.json`). The format must be as follows:

```
[
  {
    "question": "What is the definition of ...?",
    "ground_truth_answer": "The text defines it as...",
    "naive_rag_answer": "Model output using Naive RAG...",
    "contextual_retrieval_answer": "Model output using Contextual Retrieval..."
  },
  {
    "question": "Explain the concept of...",
    "ground_truth_answer": "...",
    "naive_rag_answer": "...",
    "contextual_retrieval_answer": "..."
  },
  ...
]
```

Ensure that each of your 20 QA pairs is included in this JSON file. This comparison data is a critical part of your deliverable. (0.5 point)